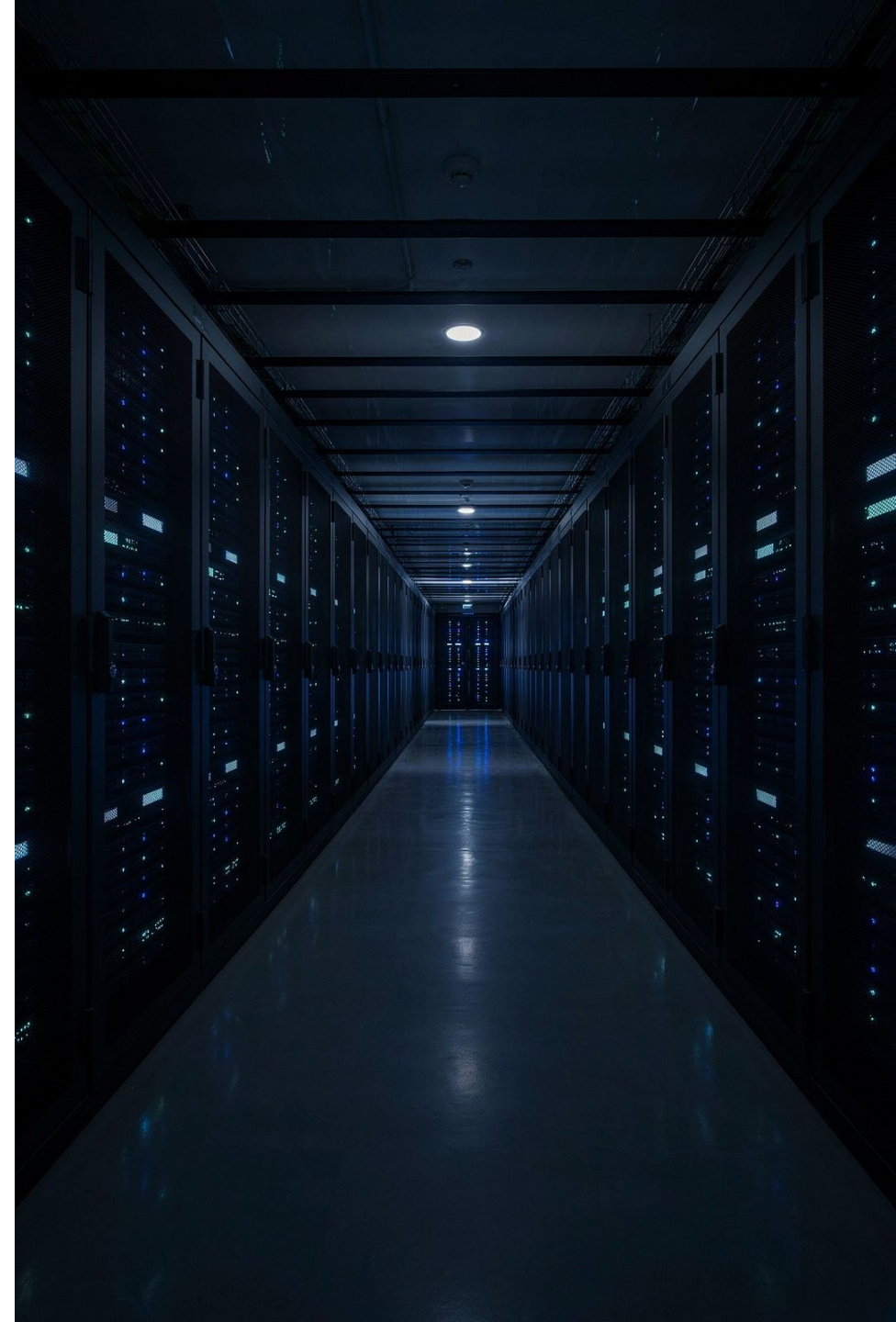


Anti-Entropy Mechanisms for Single-Digit Minute Latency in Cloud-Scale Security Platforms

A 30-minute beginner talk on building security platforms that are both fast and correct.

Rahul Kizhakkepachilamakol

Principal Software Engineer, Amazon Web Services



Meet the Speaker

**Rahul
Kizhakkepachilamakol**

Principal Software
Engineer, Amazon Web
Services

15+ years building large-scale distributed systems across cloud, e-commerce, and security-critical platforms.

- **Cloud-Scale Distributed Systems**
- **Vulnerability Management & Security Data Platforms**
- **High-Throughput Ingestion & NoSQL Architectures**
- **AI-Based & Agentic Software Development**





Why Security Finding Latency Matters

Cloud environments continuously surface vulnerability findings across compute, containers, storage, networking, identity, and application layers. A finding only has value when it reaches the teams who can act on it — fast.

Exposure Time

Delayed findings extend the window of active risk.

Incident Response

Late data slows detection, triage, and containment.

Compliance Risk

Gaps in finding visibility can trigger regulatory violations.

Remediation Backlog

Slow delivery creates compounding backlogs of unresolved issues.

 Minimizing the time between **finding generation** and **finding availability** is a critical platform requirement for security-sensitive organizations.

The Distributed Systems Challenge



Cloud-scale security platforms span multiple regions, independent data stores, distributed compute, and are subject to network partitions and transient service failures.

We Want All Three But Can't Always Have Them

How do we deliver findings quickly *and* guarantee eventual correctness when failures occur?

Fast Delivery

Findings reach consumers with minimal delay.

High Availability

The platform responds even during partial failures.

Strong Correctness

Data is accurate and complete across all nodes.

Why Streaming Alone Is Not Enough

Streaming is the right primary path for low-latency delivery. But it cannot guarantee correctness on its own.



Streaming Provides Speed

Most findings arrive quickly under normal conditions.

Anti-Entropy Provides Confidence

When consumers fail, partitions occur, or data is partially written — streaming alone cannot recover. A secondary correctness mechanism is required.

Introducing Anti-Entropy

Anti-entropy is a distributed-systems technique that **periodically compares data across nodes** and repairs inconsistencies — rather than assuming every event was delivered perfectly.

1

Compare

Check datasets across distributed stores.

2

Detect

Identify missing or corrupted records.

3

Repair

Restore correct state from the authoritative source.

4

Verify

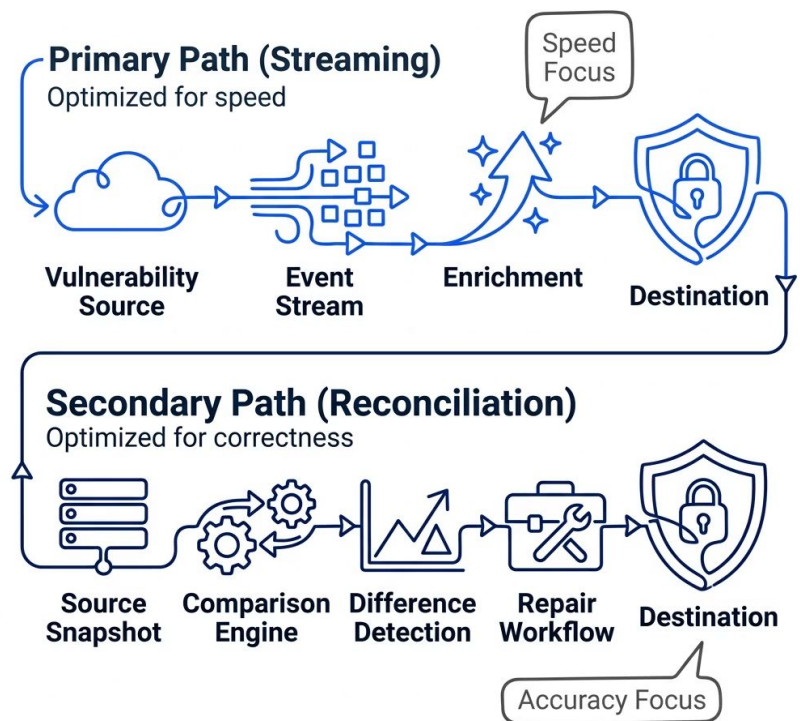
Confirm that repaired data now matches.



A vulnerability finding that **silently disappears** can be as dangerous as a system outage. Anti-entropy is your safety net.



Security-Specific Reconciliation Architecture



Two Paths, One Platform

This design lets you **optimize for speed** without permanently sacrificing correctness. The two paths complement each other.

⚡ Streaming = Fast Path

Real-time, low-latency delivery under normal conditions.

🔄 Anti-Entropy = Correctness Backstop

Periodic reconciliation catches what streaming misses.

Merkle Trees for Efficient Comparison



At cloud scale, comparing every individual finding record is expensive. **Merkle trees** make this tractable.

- ❏ Instead of scanning the full dataset, the system targets **only the sections where differences exist**.

1 Partition Data into Groups

Each leaf node represents a hash of a chunk of findings.

2 Compare Top-Level Hashes First

If they match, the underlying data is assumed identical – no further work needed.

3 Drill Into Differences Only

When hashes diverge, traverse smaller branches to isolate exactly which records are inconsistent.

Attribute-Level Hashing

A record can *exist* in both systems while still containing **wrong information**. Existence checks are not enough.

System A

Vulnerability ID: CVE-XXXX

Severity: **Critical**

Asset: Server A

System B

Vulnerability ID: CVE-XXXX

Severity: **Medium**

Asset: Server A

Same record. Different severity. Without attribute-level hashing, this discrepancy goes undetected.

What Attribute Hashing Can Catch

- Missing or null fields
- Modified severity or risk score
- Incorrect asset or ownership context
- Stale or corrupted enrichment metadata
- Inconsistent remediation status

- ❑ The reconciliation layer detects not only **missing findings** but also **silent corruption**.

Read Repair and Hinted Handoff

Two classic distributed-database techniques that prevent temporary failures from becoming permanent data loss.

Read Repair

When inconsistent versions of a finding are detected, the system compares records, identifies the authoritative value, repairs the stale copy, and records the action for auditability.

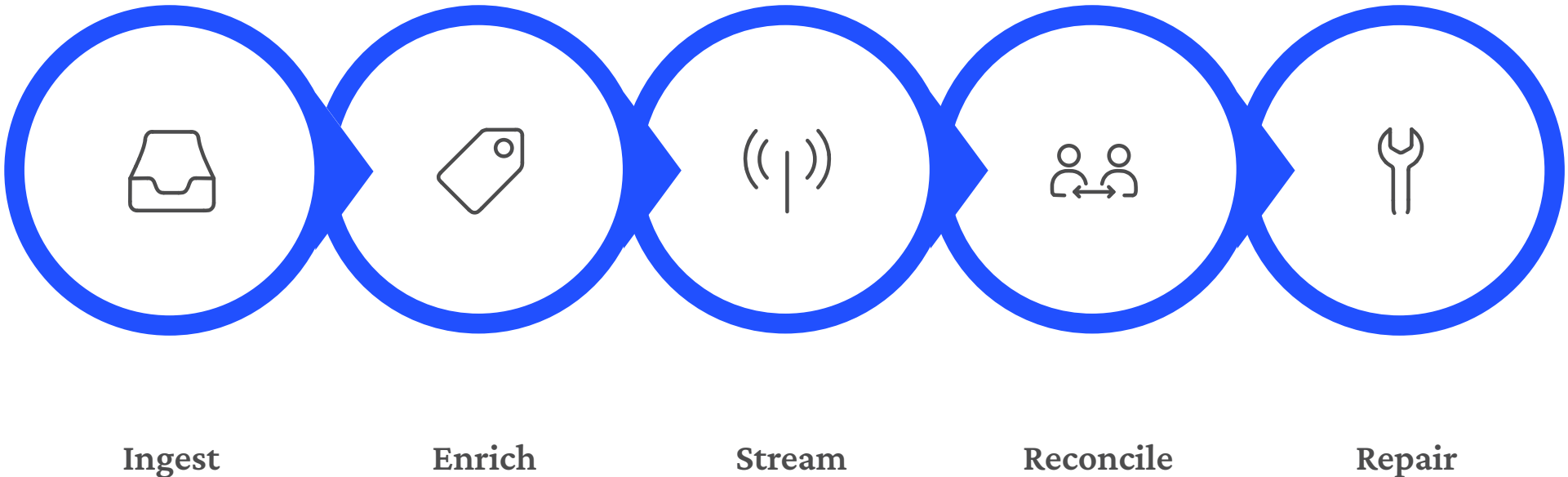
Hinted Handoff

If a destination is temporarily unavailable, the pending update is stored with a pointer to the target. When the destination recovers, delivery is retried automatically.

❏ Together, these mechanisms ensure that **infrastructure blips don't become permanent gaps** in your security data.

Streaming + Anti-Entropy Together

The full architecture combines real-time processing with periodic reconciliation for both speed and correctness.



❏ **Result:** Fast delivery under normal conditions with bounded, auditable recovery when failures occur.

Auditability and Compliance



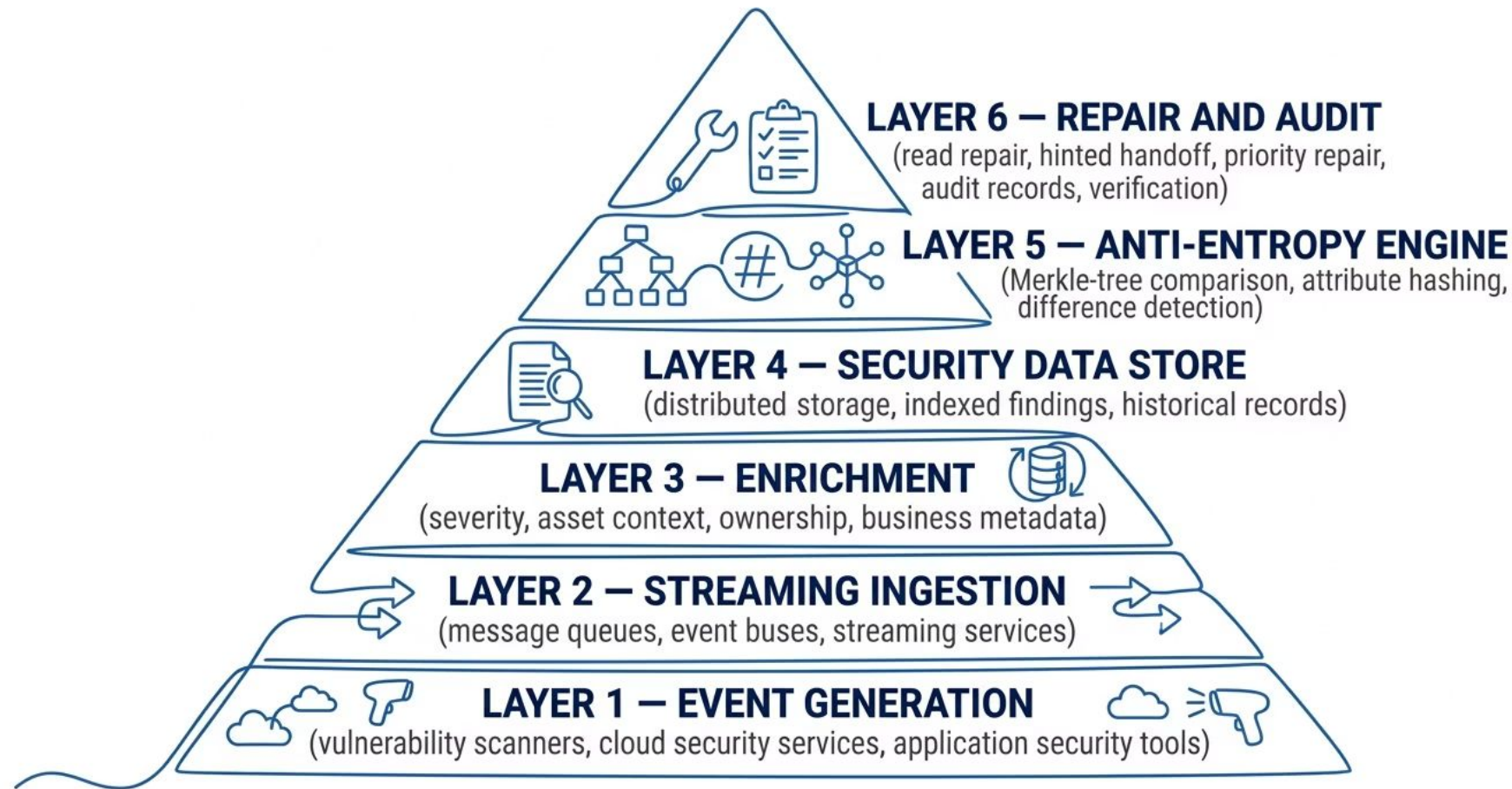
In regulated environments, the **repair itself must be auditable**. Automated corrections without records create their own compliance risk.

Every Reconciliation Action Should Capture

- ❏ **Repairing security data without recording the repair creates another compliance risk.**

- Finding identifier and original state
- Detected inconsistency and source of truth
- Repair action taken and system responsible
- Timestamp and verification result

Design Pattern for Cloud-Scale Security Platforms



Key Takeaways

Streaming Delivers Speed

Event-driven architecture gets the majority of findings to consumers quickly and efficiently.

Anti-Entropy Delivers Correctness

Assume failures will happen. Design reconciliation mechanisms to catch what streaming misses.

Compare Efficiently with Merkle Trees

Avoid expensive full-dataset scans by targeting only sections where hashes diverge.

Detect Silent Corruption

Attribute-level hashing catches inconsistencies even when records exist in both systems.

Prioritize Security Risk

Repair critical, exposed findings first when reconciliation capacity is limited.

Preserve Auditability

Every automated repair must produce a traceable, timestamped record.

High-performance security platforms should not choose between speed and correctness. A streaming-first architecture backed by anti-entropy reconciliation delivers fast findings — and continuously repairs the inconsistencies that distributed systems inevitably create.